

Socket Programming in Java

1 Introduction

Socket programming in Java enables two-way communication between distributed applications. A network application typically consists of a **client** and a **server** exchanging data via sockets.

Key Idea

Java provides `Socket` and `ServerSocket` classes for TCP, and `DatagramSocket` for UDP.

2 Socket Programming with UDP

UDP is a connectionless, fast protocol. It is suitable for applications like video streaming or real-time messaging.

2.1 UDP Client in Java

Listing 1: UDP Client

```
1  import java.net.*;
2
3  public class UDPClient {
4      public static void main(String args[]) throws Exception {
5          DatagramSocket clientSocket = new DatagramSocket();
6          InetAddress IPAddress = InetAddress.getByName("localhost");
7
8          byte[] sendData = new byte[1024];
9          byte[] receiveData = new byte[1024];
10
11         String sentence = "hello server";
12         sendData = sentence.getBytes();
13
14         DatagramPacket sendPacket = new DatagramPacket(sendData,
15             sendData.length, IPAddress, 9876);
16         clientSocket.send(sendPacket);
17
18         DatagramPacket receivePacket = new DatagramPacket(receiveData,
19             receiveData.length);
20         clientSocket.receive(receivePacket);
```

```

20         String modifiedSentence = new String(receivePacket.getData())
21         ;
22         System.out.println("FROM SERVER: " + modifiedSentence);
23         clientSocket.close();
24     }

```

2.2 UDP Server in Java

Listing 2: UDP Server

```

1  import java.net.*;
2
3  public class UDPServer {
4      public static void main(String args[]) throws Exception {
5          DatagramSocket serverSocket = new DatagramSocket(9876);
6          byte[] receiveData = new byte[1024];
7          byte[] sendData;
8
9          System.out.println("UDP Server is running...");
10
11         while (true) {
12             DatagramPacket receivePacket = new DatagramPacket(
13                 receiveData, receiveData.length);
14             serverSocket.receive(receivePacket);
15             String sentence = new String(receivePacket.getData());
16
17             String capitalizedSentence = sentence.toUpperCase();
18             sendData = capitalizedSentence.getBytes();
19
20             InetAddress IPAddress = receivePacket.getAddress();
21             int port = receivePacket.getPort();
22
23             DatagramPacket sendPacket = new DatagramPacket(sendData,
24                 sendData.length, IPAddress, port);
25             serverSocket.send(sendPacket);
26         }
27     }
28 }

```

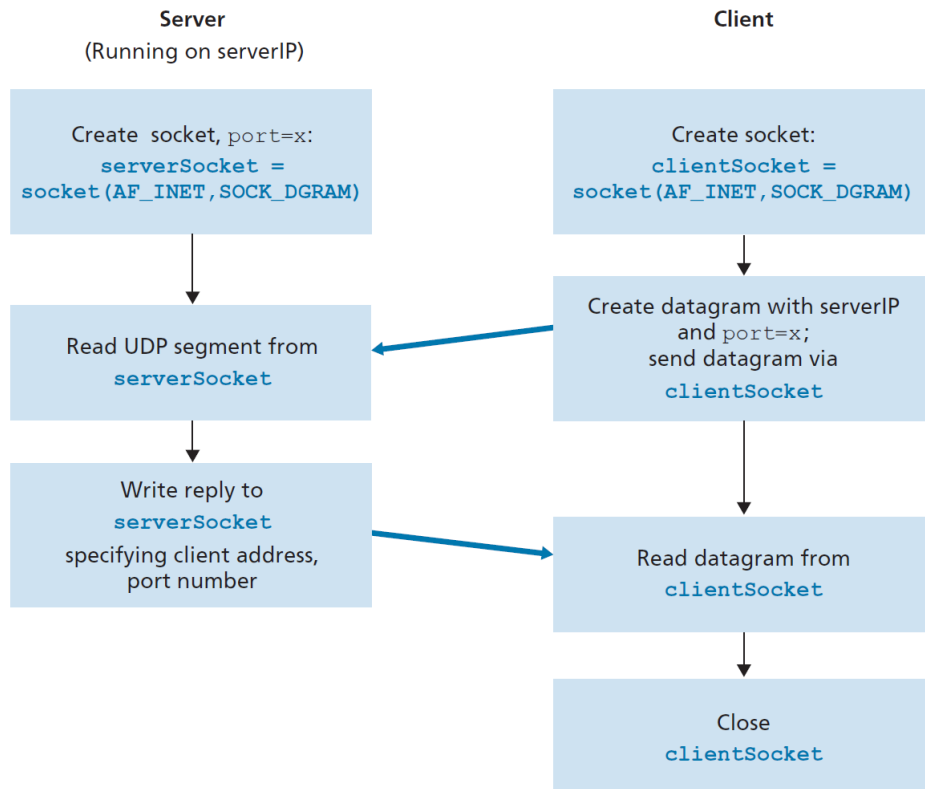


Figure 1: UDP Message Flow

3 Socket Programming with TCP

TCP provides reliable, ordered, and connection-oriented communication.

3.1 TCP Client in Java

Listing 3: TCP Client

```

1  import java.io.*;
2  import java.net.*;
3
4  public class TCPCClient {
5      public static void main(String argv[]) throws Exception {
6          String sentence = "hello server";
7          String modifiedSentence;
8
9          Socket clientSocket = new Socket("localhost", 6789);
10
11         DataOutputStream outToServer = new DataOutputStream(
12             clientSocket.getOutputStream());
13         BufferedReader inFromServer = new BufferedReader(new
14             InputStreamReader(clientSocket.getInputStream()));
15
16         outToServer.writeBytes(sentence + '\n');
17         modifiedSentence = inFromServer.readLine();
18     }
19 }

```

```

17         System.out.println("FROM SERVER: " + modifiedSentence);
18         clientSocket.close();
19     }
20 }

```

3.2 TCP Server in Java

Listing 4: TCP Server

```

1  import java.io.*;
2  import java.net.*;
3
4  public class TCPServer {
5      public static void main(String argv[]) throws Exception {
6          String clientSentence;
7          String capitalizedSentence;
8
9          ServerSocket welcomeSocket = new ServerSocket(6789);
10         System.out.println("TCP Server is running...");
11
12         while (true) {
13             Socket connectionSocket = welcomeSocket.accept();
14
15             BufferedReader inFromClient = new BufferedReader(new
16                 InputStreamReader(connectionSocket.getInputStream()));
17             DataOutputStream outToClient = new DataOutputStream(
18                 connectionSocket.getOutputStream());
19
20             clientSentence = inFromClient.readLine();
21             capitalizedSentence = clientSentence.toUpperCase() + '\n';
22             ;
23             outToClient.writeBytes(capitalizedSentence);
24         }
25     }
26 }

```

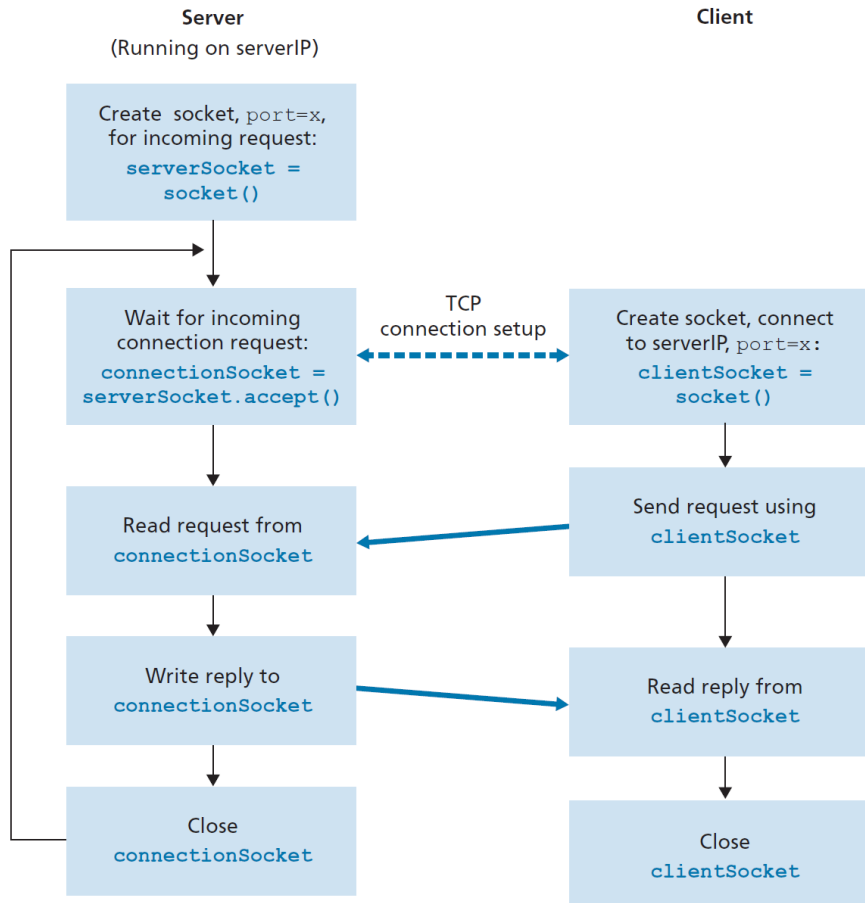


Figure 2: TCP Client-Server Communication Flow

4 Comparison: UDP vs TCP

Feature	UDP	TCP
Connection	Connectionless	Connection-oriented
Reliability	Unreliable	Reliable
Speed	Faster, low overhead	Slower, more overhead
Best For	Real-time apps, DNS, streaming	Web, Email, File Transfer

5 Conclusion

Java sockets allow building powerful distributed systems. Choice of protocol (TCP vs UDP) depends on whether the application prioritizes **speed** or **reliability**.

Pro Tip

Experiment by modifying the server to handle multiple clients concurrently using threads.